

Cheat Sheet

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

This cheat sheet distills the foundational ideas of ECE 410/510, Hardware for AI and ML (Spring 2026), from the full term of lectures. Part 1 covers the concepts worth truly internalizing. Part 2 is a quick glossary of supporting ideas. Part 3 is practical advice for staying current and entering a shifting job market. Each entry carries a numbered marker that points to a key peer-reviewed source or resource in the bibliography at the end. The tools you use will change; the foundations will not.

Part 1: Foundational concepts to internalize

1. Hardware–software co-design [1]

Definition. The concurrent design of hardware and software across all layers of the compute stack, rather than treating each as fixed for the other.

Why it matters. It is the organizing idea of the whole course. AI workloads reach acceptable performance, power, and cost only when the algorithm, compiler, and silicon are shaped together; optimizing one layer alone leaves most of the gain unrealized.

2. Moore's law, Dennard scaling, and their end [2]

Definition. Moore's law is the historical doubling of transistor density roughly every two years; Dennard scaling was the matching drop in power per transistor that has now stalled.

Why it matters. The slowdown of these trends is why general-purpose CPUs can no longer ride raw scaling, which forces the move to specialized AI hardware that this course is about.

3. The memory wall and data locality [3]

Definition. The widening gap between how fast processors compute and how fast memory can feed them, so moving data, not computing on it, dominates time and energy.

Why it matters. Off-chip DRAM access costs on the order of 100x the latency and 170x the energy of on-chip SRAM, so nearly every AI accelerator is an effort to keep data close to compute.

4. Matrix multiply as the core operation (MAC, dot product, GEMM) [4]

Definition. A multiply-accumulate (MAC) is one multiply plus one add; chained MACs form dot products, and stacked dot products form general matrix-matrix multiply (GEMM), which sits under nearly every neural-network layer.

Why it matters. Fully connected layers, convolutions, and attention all reduce to GEMM, so “good at deep learning” essentially means “good at GEMM.” This is why accelerators are built around it.

5. Arithmetic intensity [5]

Definition. The ratio of compute operations to bytes moved (FLOP/byte), which indicates whether a kernel is limited by computation or by memory traffic.

Why it matters. It is the single number that predicts whether faster compute or more bandwidth will help, and it explains why GEMM (high intensity) saturates hardware while vector adds (low intensity) starve it.

6. The roofline model [5]

Definition. A visual performance model plotting achievable performance against arithmetic intensity, bounded by a flat peak-compute ceiling and a sloped memory-bandwidth ceiling that meet at the ridge point.

Why it matters. It turns “is my kernel limited by compute or memory?” into one diagram that guides both hardware design and software optimization. Modern GPUs keep pushing the ridge point rightward, deepening the memory wall.

7. Parallelism and the GPU (SIMT) [6]

Definition. Graphics processing units exploit data parallelism through the single-instruction, multiple-threads (SIMT) model, running thousands of lightweight threads to hide memory latency.

Why it matters. GPUs became the default AI engine because SIMT maps naturally onto the massive parallelism of GEMM; understanding warps, occupancy, and coalesced memory access separates fast kernels from slow ones.

8. Domain-specific architecture and the TPU [7]

Definition. A processor purpose-built for one workload class; Google's Tensor Processing Unit is the canonical example, built around a large systolic matrix unit for neural-network inference.

Why it matters. When Moore's law cannot deliver gains, specialization does. The TPU showed order-of-magnitude efficiency improvements over CPUs and GPUs for inference, validating the premise of building hardware for AI.

9. Systolic arrays and dataflow [8]

Definition. A grid of simple processing elements that rhythmically pass data to neighbors, computing matrix products with maximal data reuse; "weight-stationary" and "output-stationary" describe which operand stays put.

Why it matters. Systolic arrays minimize the memory accesses that dominate energy, which is why they sit inside TPUs and tensor cores. The choice of dataflow directly sets energy efficiency.

10. Quantization and reduced precision [9]

Definition. Representing weights and activations with fewer bits (FP16, INT8, FP4) instead of FP32, trading numerical precision for speed, memory, and energy.

Why it matters. Lower precision multiplies effective throughput and shrinks memory traffic with little accuracy loss, which is why every modern AI chip advertises INT8/FP8/FP4 support. It is the cheapest large efficiency win available.

11. Transformers and self-attention [10]

Definition. A neural architecture that replaces recurrence with self-attention, where every token weighs its relevance to every other token via scaled dot-product attention over query, key, and value matrices.

Why it matters. Transformers power modern large language models and now drive hardware requirements industry-wide. Their attention is GEMM-heavy and memory-hungry, directly shaping accelerator and memory design.

12. In-memory and analog computing [11]

Definition. Performing computation, especially matrix-vector multiply, inside the memory array itself using device physics such as Ohm's and Kirchhoff's laws in a resistive crossbar, rather than shuttling data to a separate compute unit.

Why it matters. It attacks the memory wall at its root by eliminating data movement, promising large energy gains and offering a leading "beyond von Neumann" direction that motivates emerging-device research.

13. Neuromorphic computing and spiking neural networks [12]

Definition. Hardware modeled on the brain that co-locates memory and compute and communicates with sparse, event-driven spikes instead of clocked dense arithmetic.

Why it matters. Event-driven sparsity can cut energy dramatically for the right workloads, and neuromorphic chips such as Loihi and TrueNorth are the main exploration vehicles for ultra-low-power edge intelligence.

Part 2: Supporting concepts (quick glossary)

Universal function approximation [14]

A feedforward network with a single hidden layer can approximate any continuous function, which is why neural networks are so general-purpose.

Convolutional neural network (CNN) [15]

Weight-sharing layers that exploit spatial locality in images; heavy weight reuse raises arithmetic intensity and makes CNNs hardware-friendly.

CUDA programming model [22]

NVIDIA's framework that exposes GPU parallelism through threads, blocks, and grids, with work issued as kernels.

Tensor cores and MMA [6]

Specialized GPU units that perform a small matrix multiply-accumulate per clock in mixed precision, accelerating GEMM far beyond scalar cores.

Compute kernel [22]

A single self-contained routine (matmul, convolution, softmax) submitted to hardware; a network runs as a sequence of kernel launches.

Sparsity and pruning [9]

Removing near-zero weights or activations so hardware can skip work; the payoff grows once sparsity is high enough to beat a dense implementation.

Memristor [16]

A two-terminal device whose resistance encodes state, enabling analog synaptic weights in crossbar arrays for in-memory computing.

Reservoir computing [17]

Fix a high-dimensional dynamical “reservoir” and train only a linear readout; a low-cost route to temporal processing that can be realized in physical substrates.

VLSI, ASIC, RTL, and the EDA flow [18]

Designing a custom chip by writing register-transfer-level Verilog and running synthesis, place-and-route, and verification through automated tools; modern AI chips follow this same flow.

Loihi [13]

Intel's research neuromorphic chip with asynchronous spiking cores and on-chip learning, used to study event-driven algorithms.

PPAC trade-offs [21]

Performance, power, area, and cost: the four axes every architectural decision must balance.

Energy-efficiency metrics (TOPS/W) [19]

Operations per second per watt, the headline figure of merit for AI hardware, where energy rather than raw FLOPs is the binding constraint.

Tensor [4]

A multi-dimensional array generalizing scalars, vectors, and matrices; the basic data object that frameworks map onto hardware.

Deep learning frameworks [4]

PyTorch and TensorFlow map high-level networks onto hardware primitives and vendor libraries such as cuDNN, hiding most device detail.

Emerging and beyond-CMOS devices [20]

Photonic, spintronic, phase-change, and superconducting devices explored as successors to the transistor for AI workloads.

Part 3: Staying current and preparing for a job market in flux

1. Master the foundations, not the tools. The tools you learn this term will largely be obsolete soon; the durable advantage is understanding the full stack from algorithm to transistor.
2. Learn to reason across the stack. Be able to trace a workload from PyTorch down to memory access patterns and individual MACs; co-design thinking is what hiring managers actually want.
3. Build ML fluency. Tensor operations, quantization, sparsity, and memory access patterns are non-negotiable; an engineer who treats ML as a black box cannot make good architectural calls.
4. Develop energy-aware instincts. From voltage scaling to system power budgets, optimize for joules, not just FLOPs, because power is the limiting factor at every scale.
5. Use AI-assisted EDA as an evaluator, not a button-pusher. The shift is from “run the tool” to “judge what the tool produced,” which requires enough domain depth to audit outputs.
6. Practice verification at scale. Combine formal and simulation methods for SoCs and chiplets; verification is one of the hardest open problems and a reliable source of jobs.
7. Keep a steady intake of primary sources. The field moves on a timescale of weeks; follow arXiv, the major venues (ISCA, MICRO, ISSCC, NeurIPS), and a few trusted newsletters.
8. Build things in public. Implement a small accelerator such as a systolic array or a crossbar MVM in RTL using open tools (Verilator, Yosys, OpenROAD, cocotb) and share it on GitHub.
9. Use AI tools to learn faster, but verify their outputs. Treat large language models as accelerators for understanding, never as authorities.
10. Cultivate adaptability over credentials. Employability comes from being able to pick up the next tool because you understand the principles underneath it.

Bibliography

- [1] Teich, J. (2012). Hardware/software codesign: The past, the present, and predicting the future. *Proceedings of the IEEE*, 100(Centennial Issue), 1411–1430. <https://doi.org/10.1109/JPROC.2011.2182009>
- [2] Shalf, J. (2020). The future of computing beyond Moore’s law. *Philosophical Transactions of the Royal Society A*, 378(2166), 20190061. <https://doi.org/10.1098/rsta.2019.0061>
- [3] Wulf, W. A., & McKee, S. A. (1995). Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1), 20–24. <https://doi.org/10.1145/216585.216588>
- [4] Mishra, A., Cha, J., Park, H., & Kim, S. (Eds.). (2023). *Artificial Intelligence and Hardware Accelerators*. Springer. <https://doi.org/10.1007/978-3-031-22170-5>
- [5] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76. <https://doi.org/10.1145/1498765.1498785>
- [6] Lindholm, E., Nickolls, J., Oberman, S., & Montrym, J. (2008). NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro*, 28(2), 39–55. <https://doi.org/10.1109/MM.2008.31>
- [7] Jouppi, N. P., Young, C., Patil, N., Patterson, D., et al. (2017). In-datacenter performance analysis of a Tensor Processing Unit. *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 1–12. <https://doi.org/10.1145/3079856.3080246>
- [8] Chen, Y.-H., Krishna, T., Emer, J. S., & Sze, V. (2017). Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1), 127–138. <https://doi.org/10.1109/JSSC.2016.2616357>
- [9] Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1510.00149>

- [10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008. <https://arxiv.org/abs/1706.03762>
- [11] Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R., & Eleftheriou, E. (2020). Memory devices and applications for in-memory computing. *Nature Nanotechnology*, 15, 529–544. <https://doi.org/10.1038/s41565-020-0655-z>
- [12] Mead, C. (1990). Neuromorphic electronic systems. *Proceedings of the IEEE*, 78(10), 1629–1636. <https://doi.org/10.1109/5.58356>
- [13] Davies, M., Srinivasa, N., Lin, T.-H., Chinya, G., et al. (2018). Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1), 82–99. <https://doi.org/10.1109/MM.2018.112130359>
- [14] Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- [15] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- [16] Strukov, D. B., Snider, G. S., Stewart, D. R., & Williams, R. S. (2008). The missing memristor found. *Nature*, 453, 80–83. <https://doi.org/10.1038/nature06932>
- [17] Tanaka, G., Yamane, T., Héroux, J. B., Nakane, R., Kanazawa, N., Takeda, S., Numata, H., Nakano, D., & Hirose, A. (2019). Recent advances in physical reservoir computing: A review. *Neural Networks*, 115, 100–123. <https://doi.org/10.1016/j.neunet.2019.03.005>
- [18] Kahng, A. B., Lienig, J., Markov, I. L., & Hu, J. (2011). *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Springer. <https://doi.org/10.1007/978-90-481-9591-6>
- [19] Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12), 2295–2329. <https://doi.org/10.1109/JPROC.2017.2761740>
- [20] Schneider, M. L., Jué, E. M., Pufall, M. R., Segall, K., & Anderson, C. W. (2025). A self-training spiking superconducting neuromorphic architecture. *npj Unconventional Computing*. <https://doi.org/10.1038/s44335-025-00021-9>
- [21] Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
- [22] NVIDIA. (2025). *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide>